D INFK

Nonlinear regression & Optimization I

Introduction to Machine Learning, Lecture 3

Fanny Yang

ETH zürich

Announcements

- 2-stage Project procedure
  - Stage I: To try different solutions out, submit as many attempts as you want to public dataset
  - Stage II: Official hand in of one solution (must be done before deadline!), evaluated on public and private dataset
  - We will not accept late hand-ins even if you have already submitted a solution
- Project 0 released yesterday (no deadline), ungraded
  - project.las.ethz.ch
  - for testing purposes, make sure the uploading process works for future tasks

    – no deadline extensions in future projects!
  - you upload your individual video link in step 3 “Hand in final solution”!
- First homework will be released next week

Recap of past week: Linear regression

Training Data → ML method: efficiently find $\hat{f} \in F$ → Model $\hat{f}$

ML Scientists design



Function class F	Training loss	Optimization
• linear functions	• squared loss	• closed-form

Recap of past week: Supervised learning so far

- Goal: find $\hat{f}$ that minimizes training loss, i.e. $\hat{f} = \text{argmin}_{f \in F} L(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f(x_i))$

  in the linear function class $F = \{ f_w(x) = w^\top x \mid w \in \mathbb{R}^d \}$ *(setting $w_0 = 0$ for simplicity)*

- Hence it was equivalent to find $\hat{w}$ and set $\hat{f} = f_{\hat{w}}$

$$\hat{w} = \text{argmin}_{w \in \mathbb{R}^d} L(w) = \text{argmin}_{w \in \mathbb{R}^d} \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f_w(x_i))$$

- So far, given data, we explicitly computed $\hat{f}, \hat{w}$ that also minimizes training loss

This week: extension of toolkit

Training Data → ML method: efficiently find $\hat{f} \in F$ → Model $\hat{f}$

ML Scientists design



Function class F	Training loss	Optimization
• linear functions	• squared loss	• closed-form
• nonlinear functions		• gradient methods

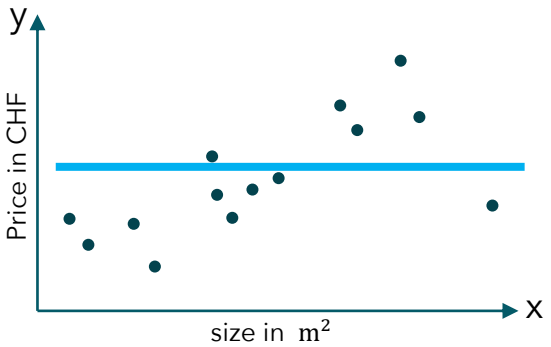
Plan for today

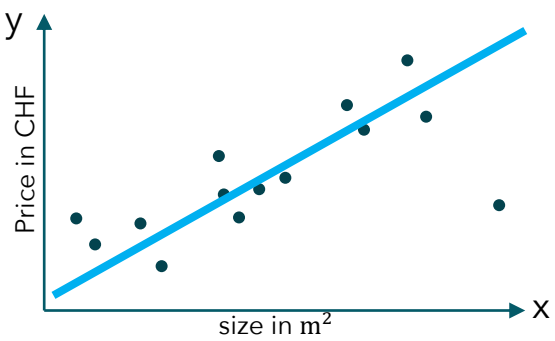
- Nonlinear regression
  - Nonlinear functions as linear functions of nonlinear features
  - Writing the loss in matrix-vector notation and finding the closed-form minimizer
- Gradient descent for general losses

Different function classes F

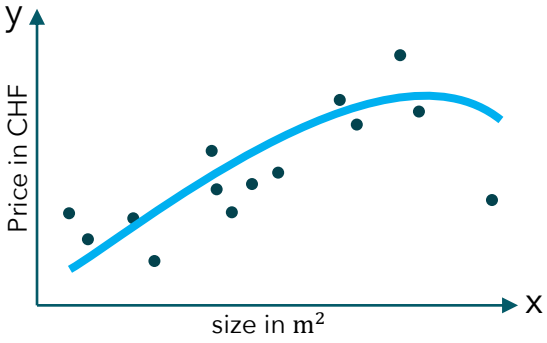
Back to 1-d example with input x: size in m^2: With what kind of functions can I fit 1-dimensional data?

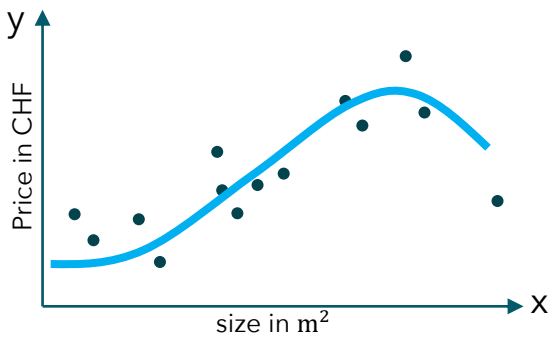
Last week: constant and linear





This week: polynomial, nonlinear functions





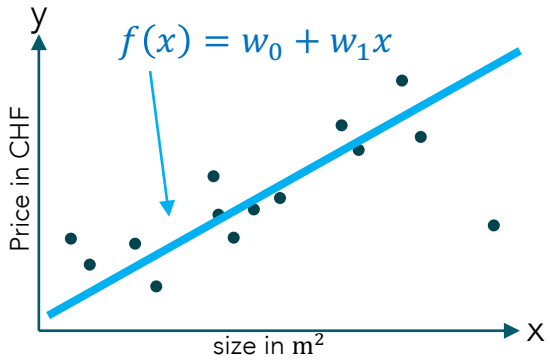
Example: polynomial functions

Function class F

- polynomial function

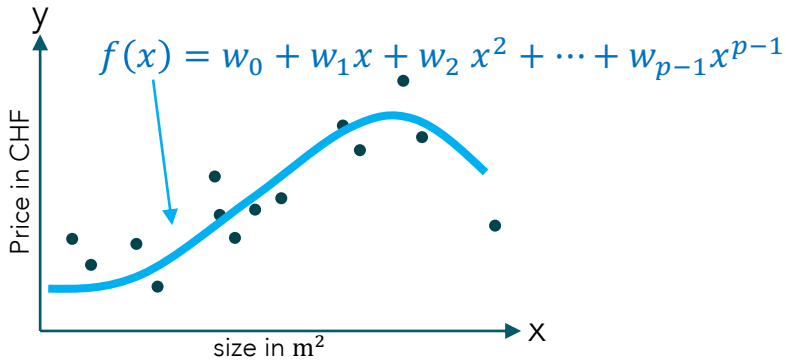
note: analogous to $\langle w, x \rangle$ being a linear function in x,
$\langle w, \phi(x) \rangle$ is linear in the feature vector $\phi(x)$ (but not linear in x)!

Both can be written as $f(x) = \sum_{j=0}^{p-1} w_j \, \phi_j(x)$ with *feature vector* $\phi(x) = \left(\phi_0(x), \ldots, \phi_{p-1}(x)\right) \in \mathbb{R}^p$



from linear

to higher deg. polynomial

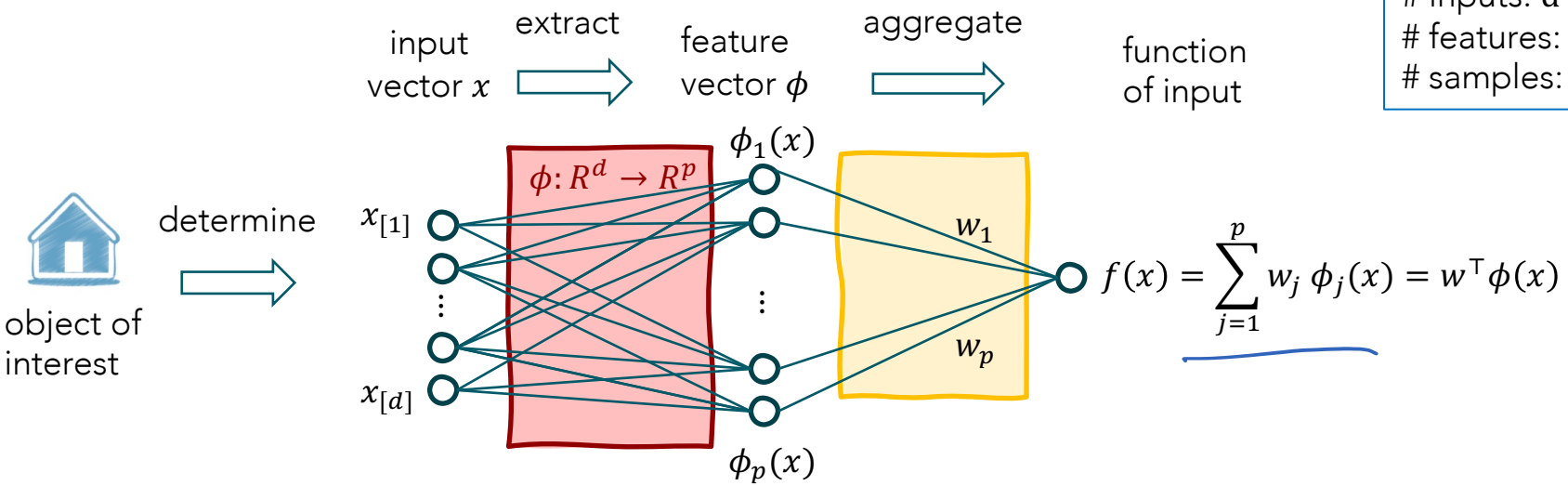


Linear function with $\phi_0(x) = 1, \phi_1(x) = x$

uses additional monomials $\phi_2(x) = x^2 \ldots \phi_{p-1}(x) = x^{p-1}$

Linear models are more powerful than you think

Many types of nonlinear functions of input vector x can be written as



- Some ML models fit data with fixed mappings ϕ: Linear regression, polynomial and kernel regression
- Others also learn the mappings ϕ when fitting the data: neural networks

More examples for nonlinear feature maps ϕ

Function class F
• nonlinear functions

Fixed feature function classes : $F_\phi = \{f\colon f(x) = \sum_{j=1}^{p} w_j\, \phi_j(x) \text{ for } w \in \mathbb{R}^p\}$
for some nonlinear functions $\phi_1, \ldots, \phi_p$

Examples: Multi-dimensional polynomials

x ⇨

Feature $\phi_1(x) = 1$
Feature $\phi_2(x) = x_{[1]}$
Feature $\phi_3(x) = x_{[1]}x_{[2]}$
...
Feature $\phi_p(x) = x_{[1]}x_{[2]}^2x_{[3]}^5$

Trigonometric basis (in tutorial)

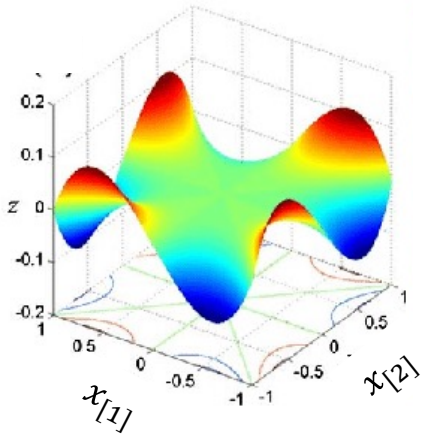
x ⇨

Feature $\phi_1(x) = 1$
Feature $\phi_2(x) = \sin 2\pi x$
...
Feature $\phi_p(x) = \sin 2\pi p x$

*Concrete example in homework next week

Squared loss for linear functions of p features

Function class F	Loss function	Optimization
• polynomial function	• squared loss	• closed-form



A polynomial of 2d inputs

We again seek the minimizer of the square loss

$$\hat{f} = \underset{f \in F_\phi}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f(x_i)\right)^2$$

$f(x) = \langle w, \phi(x) \rangle$

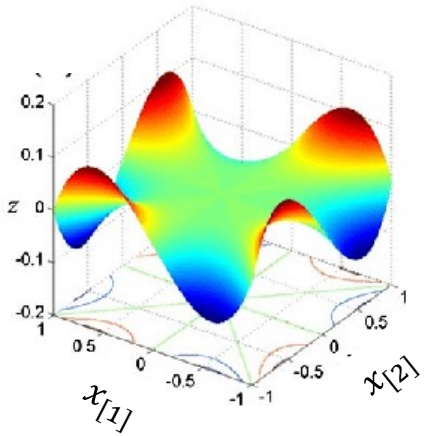
$\hat{w} = \underset{w \in \mathbb{R}^p}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} (y_i - \langle w, \phi(x_i) \rangle)^2$

$\phi(x_i) = \begin{pmatrix} \phi_1(x_i) \\ \vdots \\ \phi_p(x_i) \end{pmatrix}$

$\hat{f} = f_{\hat{w}}(x) = \langle \hat{w}, \phi(x) \rangle$

Squared loss for linear functions of p features

Function class F	Loss function	Optimization
• polynomial function	• squared loss	• closed-form



A polynomial of 2d inputs

We again seek the minimizer of the square loss

$$\hat{f} = \underset{f \in F_\phi}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \big(y_i - f(x_i)\big)^2$$

this time in some fixed feature function space F_ϕ

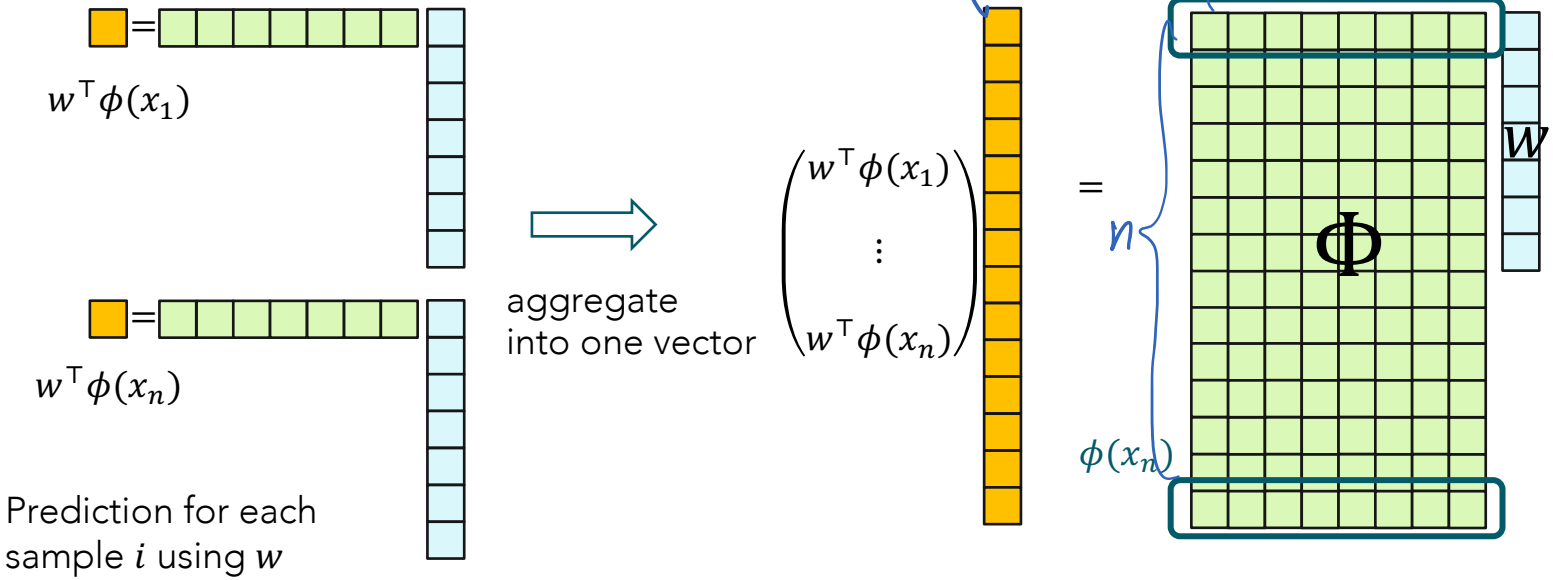
We can again parameterize the loss using $f(x) = \langle w, \phi(x) \rangle$, find minimizing parameter vector $\hat{w}$ and set $\hat{f} = f_{\hat{w}}$

$$\hat{w} = \underset{w \in \mathbb{R}^p}{\operatorname{argmin}} L(w) = \frac{1}{n} \sum_{i=1}^{n} (y_i - w^\top \phi(x_i))^2$$

Fitting nonlinear functions

Similar to linear regression, we can rewrite the training loss L in matrix vector notation

$$L(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - w^\top \phi(x_i))^2 = \frac{1}{n}||y - \Phi w||_2^2$$



Closed-form minimizer for nonlinear regression

$\Phi^T \Phi \in \mathbb{R}^{p \times p}$

- Recap: for linear regression we derived that minimizer $\hat{w}$ of $\frac{1}{n}||y - Xw||^2$ has to satisfy: $X^T y = X^T X \hat{w}$ by computing stationary point or using a projection argument
- → When $X^T X$ invertible, we have $\hat{w} = (X^T X)^{-1} X^T y$ and finally obtain $\hat{f}(x) = \langle \hat{w}, x \rangle$

- Inspecting the new training loss $\frac{1}{n}||y - \Phi w||^2$, we can just replace X by Φ and analogously obtain $\hat{w} = (\Phi^T \Phi)^{-1} \Phi^T y$ and the minimizing nonlinear function

$$\hat{f}(x) = \langle \hat{w}, \phi(x) \rangle = \langle (\Phi^T \Phi)^{-1} \Phi^T y, \phi(x) \rangle$$

- Issues with both closed-form solutions:
  - **High cost:** generally have to compute inverse of a $p \times p$ (or $d \times d$) matrix!
  - **Lack of generality**: How to find $\hat{f}/\hat{w}$ *for arbitrary loss* when closed form can't be directly calculated?

This week: extension of toolkit

Training Data → ML method: efficiently find $\hat{f} \in F$ → Model $\hat{f}$

ML Scientists design



Function class F	Training loss	Optimization
• linear functions	• squared loss	• closed-form
• nonlinear functions		• gradient methods

Plan for today

- Nonlinear regression
- Gradient descent for general losses
  - Warmup with 1d case: directions, stepsize, stopping criterion
  - Multi-d inputs:
    i. visualization of losses in 2d
    ii. negative gradient as steepest descent direction

Iterative procedure on a high level – for 1d

General iterative algorithm to minimize $L(w)$

Start at initial $w^{now} = w^{start}$

At each step: $w^{next} = w^{now} + \tilde{\eta} \cdot v$ and set $w^{now} = w^{next}$

$\tilde{\eta}$: how far to move

v: update direction

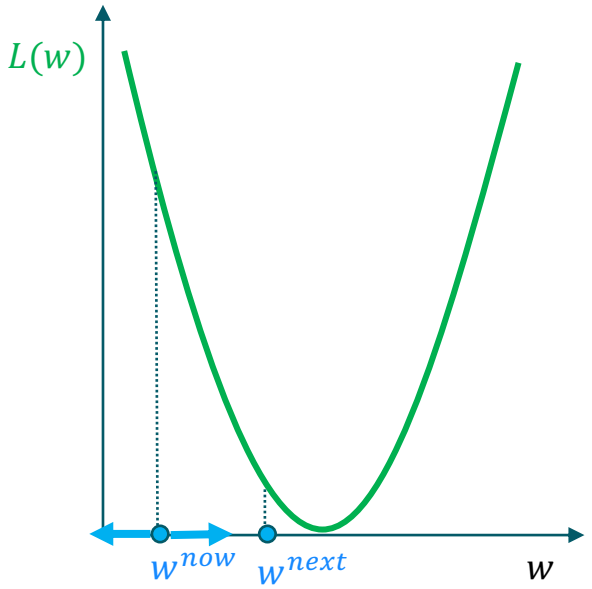
When can't improve much more, stop and output $\hat{w} = w^{now}$

At each step, depending on L, w^{now}, the questions are:

Q1: Which direction v to choose?

Q2: How far ($\tilde{\eta} > 0$) to go in that direction?

Q3: When should we stop?



Setting now: Minimize $L(w)$ with scalar w

(e.g. linear regression in 1-d)

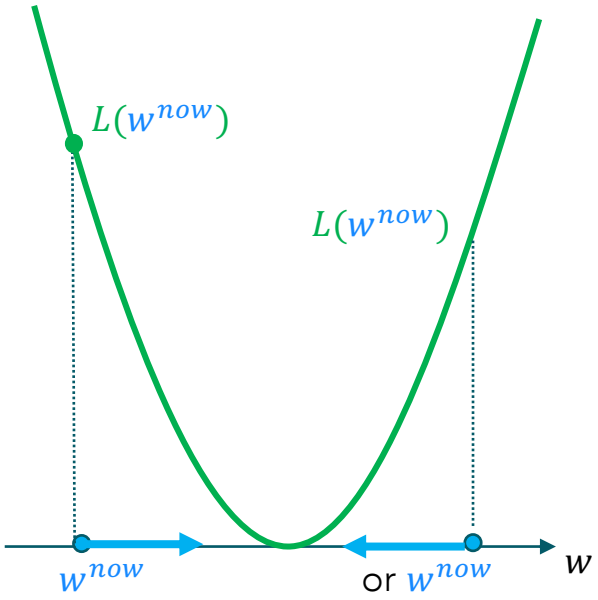
Iteratively minimizing a 1-d function – **Q1**: direction

At each step: $w^{next} = w^{now} + \tilde{\eta}\, v$

- **Q1**: which direction v (+ or -) to move next to decrease L?
- Common sense: if $L(w)$ decreases as w increases
  (i.e., if $L'(w) < 0$) ⇒ increase w (and vice versa)

→ Mathematically speaking, choose direction

$$v = -\text{sign}\big(L'(w)\big) = -\frac{L'(w)}{|L'(w)|}$$



Iteratively minimizing a 1-d function – **Q2**: how far

At each step: $w^{next} = w^{now} - \tilde{\eta}\ \mathrm{sign}\big(L'(w)\big) = w^{now} - \tilde{\eta}\dfrac{L'(w^{now})}{|L'(w^{now})|}$

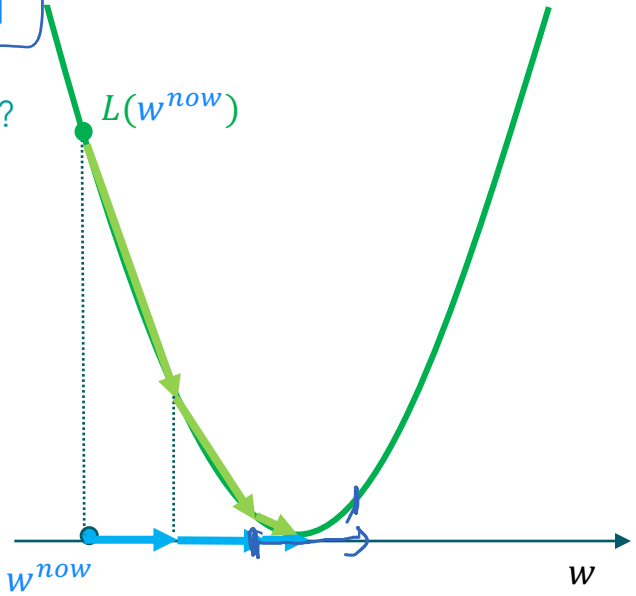
- **Q2**: How far should I go in that direction, i.e. how large $\tilde{\eta} > 0$?
- Have to decrease $\tilde{\eta}$ as slope $|L'(w^{now})|$ gets smaller

  ⇒ it cannot be constant!
- e.g. choose $\tilde{\eta}$ proportional to $|L'(w^{now})|$, such as

  $\tilde{\eta} = \eta|L'(w^{now})|$ for some constant η, so that we have

$$w^{next} = w^{now} - \eta L'(w^{now})$$

commonly called learning rate / stepsize



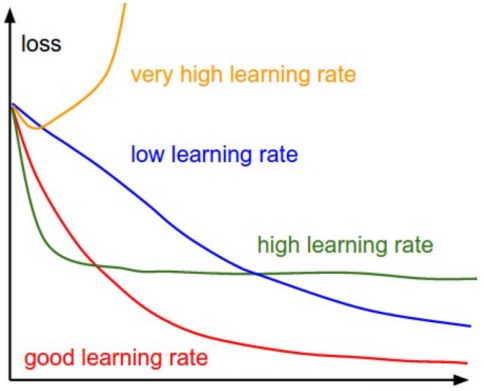
Iteratively minimizing a 1-d function – **Q2**: how far

learning rate / step size

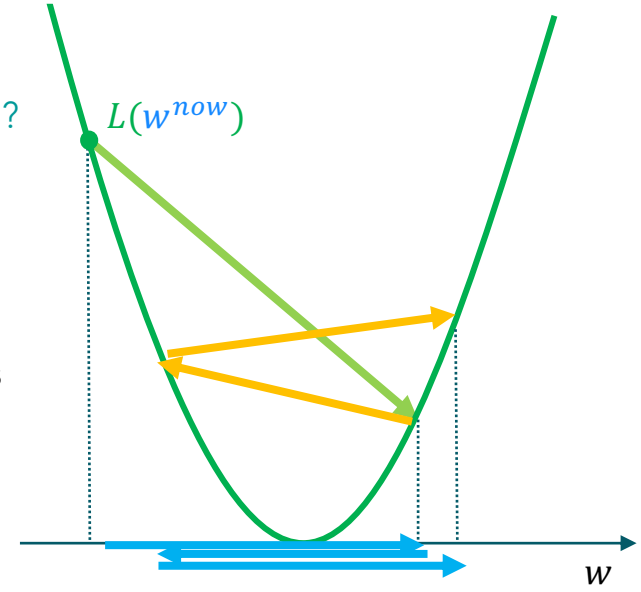
At each step: $w^{next} = w^{now} - \eta\, L'(w^{now})$ (this is called gradient descent update in 1d!)

- **Q2**: How far should I go in that direction, i.e. how large $\tilde{\eta} > 0$?

  η needs to be small enough, else too slow or diverging

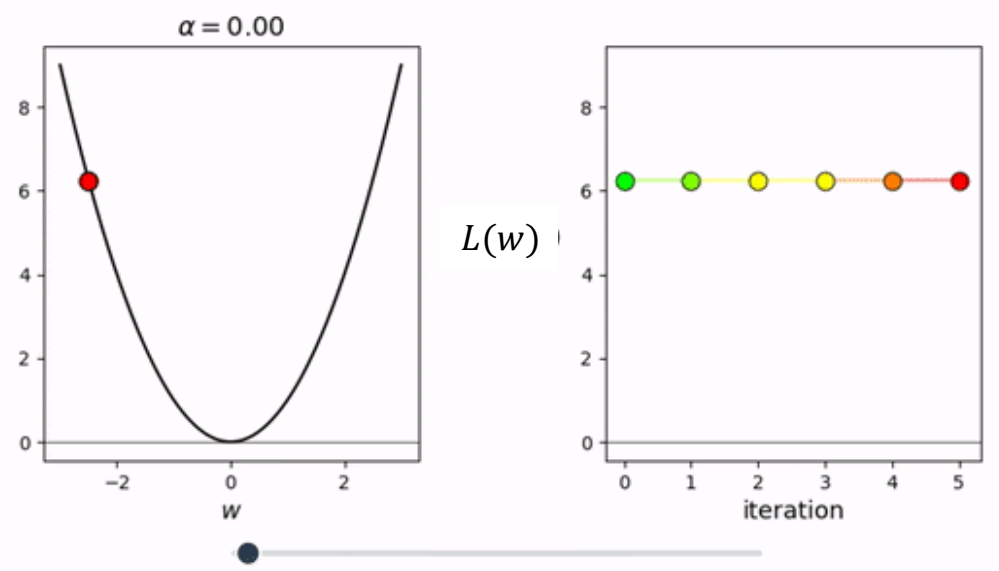


too large steps



Iteratively minimizing a 1-d function – **Q2**: how far

As constant stepsize η (in this animation α) increases

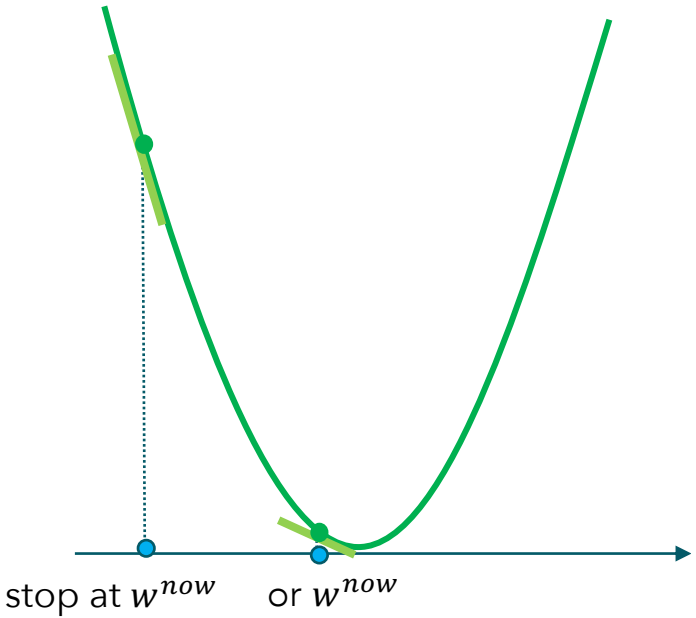


Loss value as a function of w

Loss value as time progresses

Source: https://github.com/jermwatt/machine_learning_refined Chapter 3.5

Iteratively minimizing a 1-d function – **Q3**: stop when?



Q3: Want to stop "when it can't improve much"

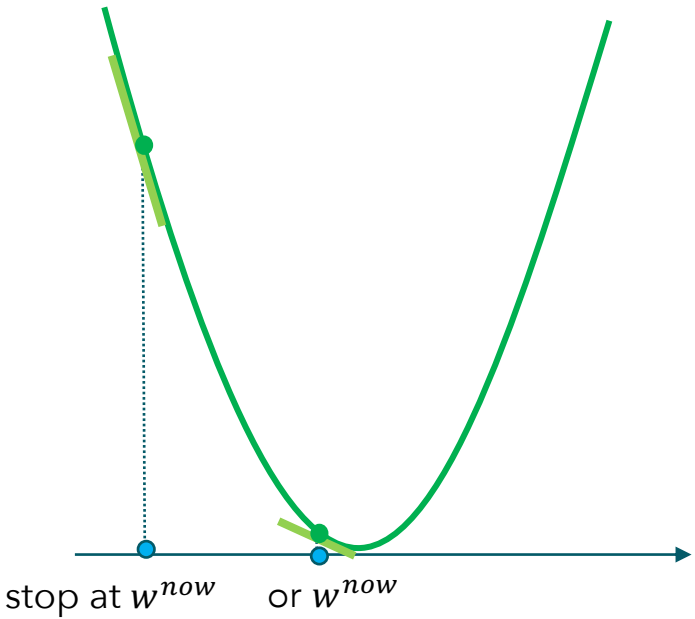
At a point w^{now}, how do you know if you can still improve?



Discuss with your neighbor.

- $|L'(w^{now})| < \varepsilon$
- $\tilde{\eta} = \eta\,|L'(w^{now})| < \varepsilon$
- $|w^{previous} - w^{now}| < \varepsilon$

Iteratively minimizing a 1-d function – **Q3**: stop when?



Q3: Want to stop "when it can't improve much"

At a point w^{now}, how do you know if you can still improve?



Discuss with your neighbor.

Some solutions in practice:

- Could look at change $L(w^{now}) - L(w^{previous})$ or $w^{now} - w^{previous}$
- or the derivative (expected change for next step) $L'(w^{now}) < 10^{-5}$

  → This also means we stop at a point close to stationary point
- Could also check Hessian …

Iterative procedure on a high level – summary for 1d

General iterative algorithm to minimize $L(w)$

Start at initial w^{start}

At each step: $w^{next} = w^{now} + \tilde{\eta}\left[-sign\left(L'(w^{now})\right)\right]$ (how far: $\tilde{\eta}$; direction: $-sign\left(L'(w^{now})\right)$)

If e.g. $|L(w^{next}) - L(w^{now})| > \epsilon$, set $w^{now} = w^{next}$ & continue

Else, stop and output $\hat{w} = w^{now}$

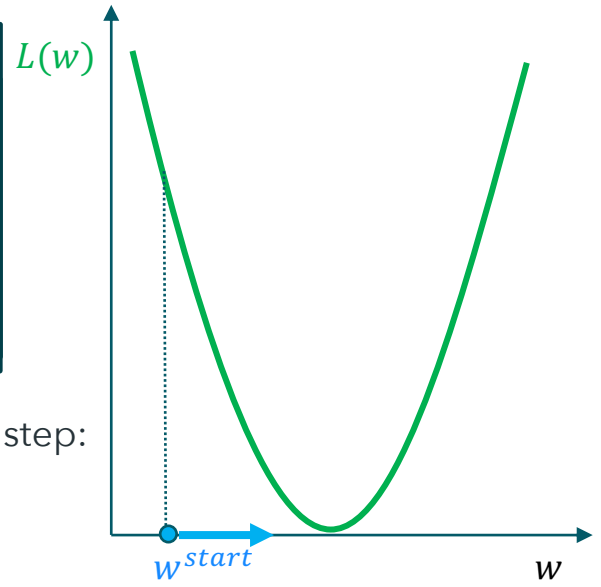
… incorporating following answer to original questions for each step:

Q1: Which direction v? → sign of negative derivative at w^{now}

Q2: How far ($\tilde{\eta} > 0$) to go in that direction? → small enough

e.g. set $\tilde{\eta}\left[-sign\left(L'(w^{now})\right)\right] = -\eta L'(w^{now})$ for not too large η

Q3: When to stop? → e.g. when $|L(w^{next}) - L(w^{now})| \leq \epsilon$



Minimize $L(w)$ with scalar w

(e.g. linear regression in 1-d)

How about for multiple $d > 1$ inputs?

General iterative algorithm to minimize $L(w)$

Start at initial w^{start}

At each step $w^{next} = w^{now} + \tilde{\eta} \cdot v$ (how far to move; update direction depending on L, w^{now})

If e.g. $|L(w^{next}) - L(w^{now})| > \epsilon$, set $w^{now} = w^{next}$ & continue

Else, stop and output $\hat{w} = w^{now}$

At each step, depending on L, w^{now}, the questions are:

Q1: Which direction v? ← many more than two to choose from!

Q2: How far $(\tilde{\eta} > 0)$ to go in that direction? → not too far

Q3: When to stop? → e.g. when $|L(w^{next}) - L(w^{now})| \leq \epsilon$

} intuition unchanged

Plan for today

- Nonlinear regression
- Gradient descent for general losses
  - Warmup with 1d case: directions, stepsize, stopping criterion
  - Multi-d inputs:
    i. visualization of losses in 2d
    ii. negative gradient as steepest descent direction

How about for multiple $d > 1$ inputs?

General iterative algorithm to minimize $L(w)$

Start at initial w^{start}

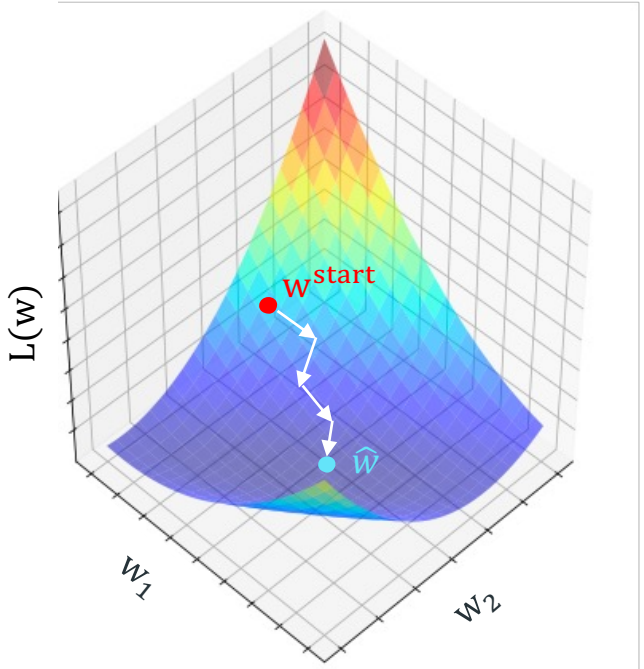
At each step $w^{next} = w^{now} + \tilde{\eta} \cdot v$ (how far to move; update direction depending on L, w^{now})

If e.g. $|L(w^{next}) - L(w^{now})| > \epsilon$, set $w^{now} = w^{next}$ & continue

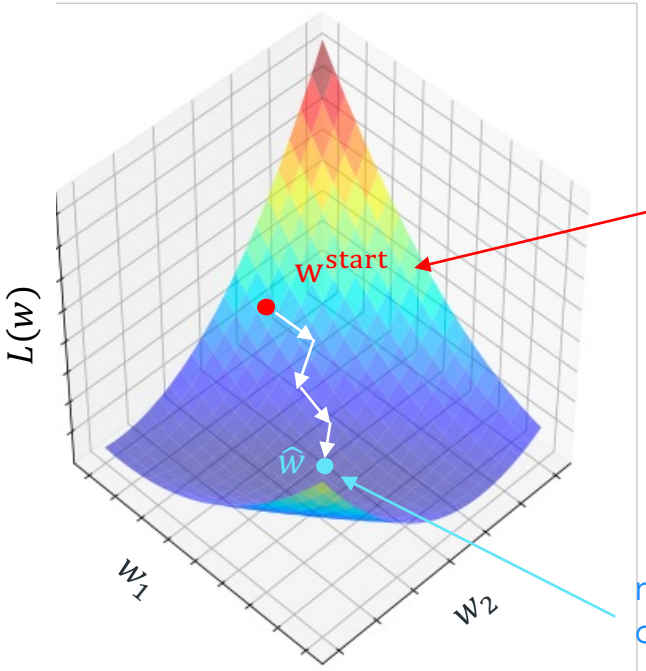
Else, stop and output $\hat{w} = w^{now}$

Again visualization beyond $d = 2$ is hard,

hence our visualizations are for $w \in \mathbb{R}^2$



Visualization of optimization landscapes in 2d

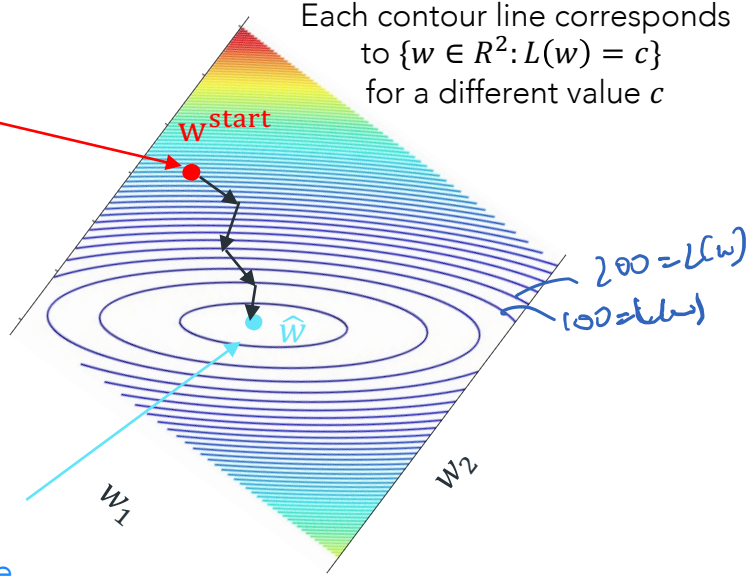


3d loss function plot

Initial (random) point

result of some iterative optimization procedure

Each contour line corresponds to $\{w \in R^2 : L(w) = c\}$ for a different value c

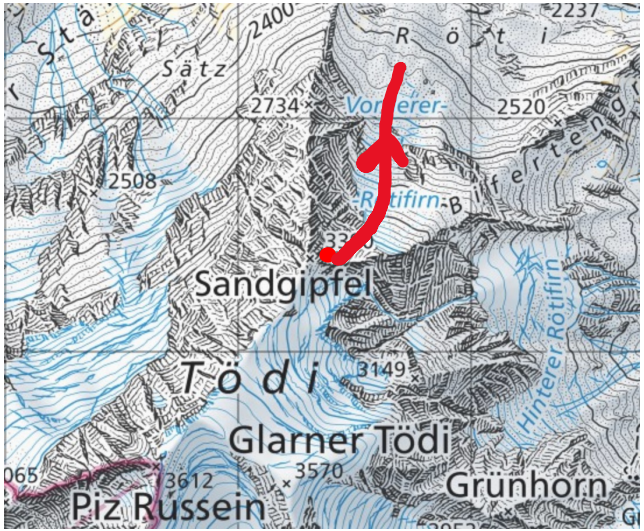


2d contour plot

Loss minimization from a hiker's perspective

Which direction would you follow if you wanted to go down the shortest possible route?

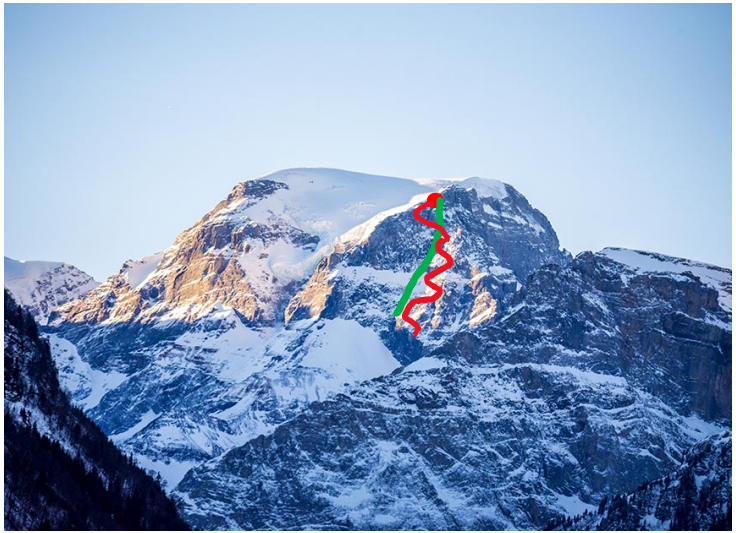


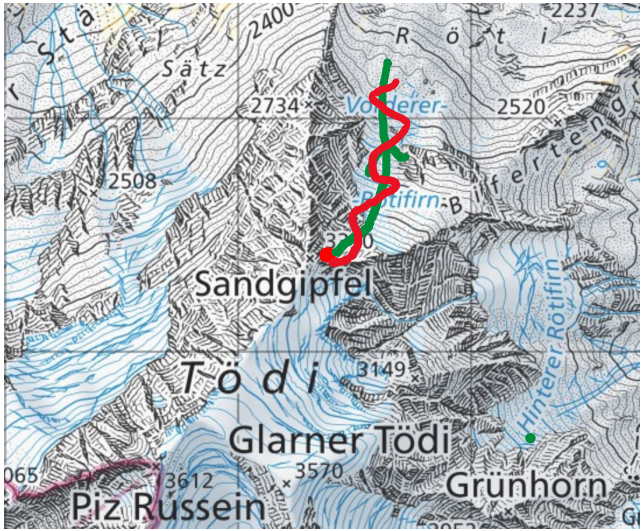


→ the steepest descent direction which is orthogonal to the contour lines!

Loss minimization from a hiker's perspective

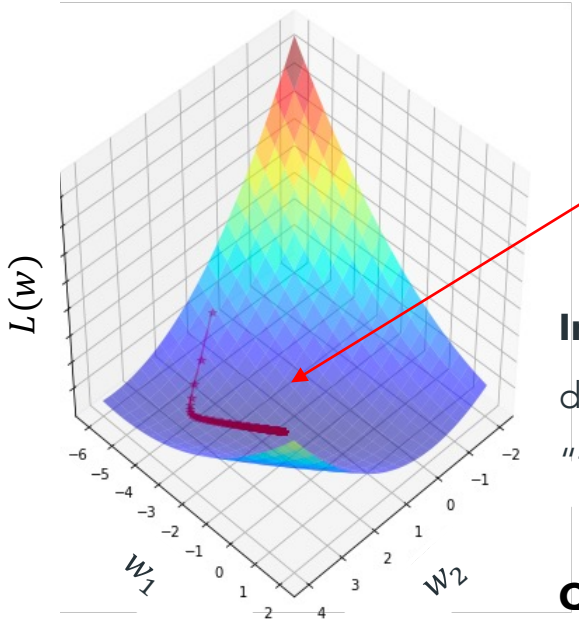
Which direction would you follow if you wanted to go down fast but also technically more easily?



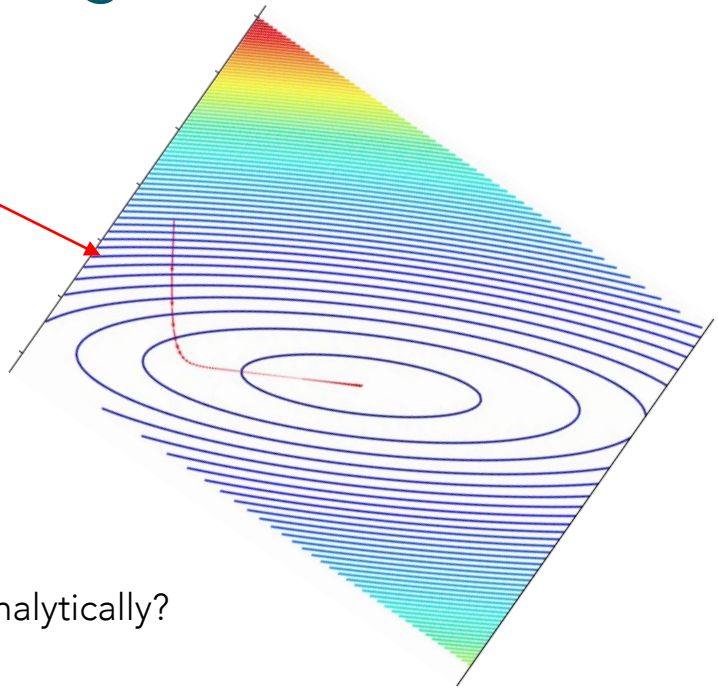


→ At least on average following the steepest descent direction (stochastic methods – next lecture)

Steepest descent direction and the gradient



Optimization path of steepest descent



Intuitively: Steepest descent direction v is the direction in which "function decreases most rapidly"

Question: How do we compute it analytically?

We now derive: at a given point w^{now} for a loss L, the steepest descent direction v is in the direction of the negative gradient $-\nabla_w L(w^{now})$

Plan for today

- Nonlinear regression
- Gradient descent for general losses
  - Warmup with 1d case: directions, stepsize, stopping criterion
  - Multi-d inputs:
    i. visualization of losses in 2d
    ii. derive that steepest descent is in the direction of the negative gradient
      a. graphically using orthogonality to contour lines
      b. arithmetically using linear approximation

Argument using contour lines in 2d

Entire line of reasoning (for 2 dimensions)

1. Intuition (already established): contour lines ⊥ steepest descent

2. By definition, we can derive that direction tangent to contour line ⊥ gradient

3. Combining 1 & 2 ⟹ gradient is ∥ (parallel to) steepest descent direction

Left to do Step 2: How to characterize direction that is tangent to contour line at a point w^{now}?

In what follows, v will be candidate directions (unit-norm vectors) in which we can move from w^{now}

Step 2: direction tangent to contour line ⊥ gradient

- By definition, a tiny step α in the direction v_0 along/tangent to the contour line at a point w^{now} should result in no change i.e. $L(w^{now} + \alpha v_0) \approx L(w^{now})$, or formally, that the following limit is zero

$$\lim_{\alpha \to 0} \frac{L(w^{now} + \alpha v_0) - L(w^{now})}{\alpha \|v_0\|_2} = 0$$

$$=: \frac{\partial L}{\partial v_0}(w^{now})$$

directional derivative of L in direction v_0 at w_{now}

*Formal statement and proof in Lemma 5.3. in lecture notes

Step 2: direction tangent to contour line ⊥ gradient

- By definition, a tiny step α in the direction v_0 along/tangent to the contour line at a point w^{now} should result in no change i.e. $L(w^{now} + \alpha v_0) \approx L(w^{now})$, or formally, that the following limit is zero

$$\lim_{\alpha \to 0} \frac{L(w^{now} + \alpha v_0) - L(w^{now})}{\alpha ||v_0||} = 0$$

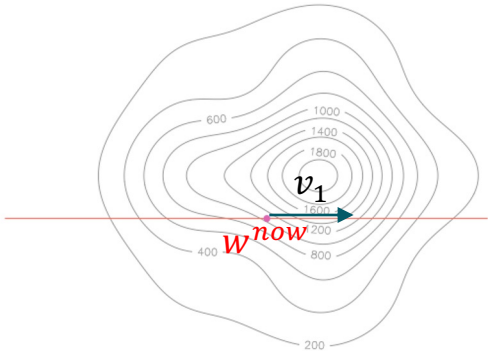
- The left-hand side is both:
  - the derivative of function $g(\alpha) = L(w^{now} + \alpha v_0)$ at $\alpha = 0$ (since $||v_0|| = 1$) and
  - the directional derivative of L at w^{now} in the direction of v_0, also denoted as $\frac{\partial L}{\partial v_0}(w^{now})$

- In the lecture notes we see in Example 2.2. that by the chain rule $\frac{\partial L}{\partial v_0}(w^{now}) = \langle \nabla_w L(w^{now}), v_0 \rangle = 0$

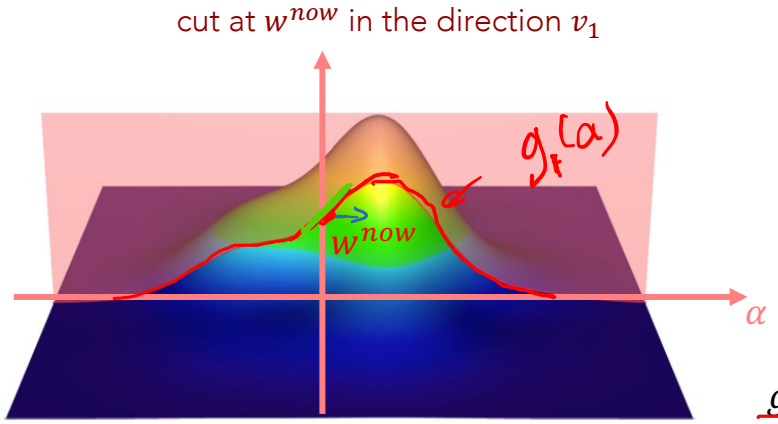
**Formal statement and proof in Lemma 5.3. in lecture notes*

Visualization of directional derivatives

Plot of function $g_1(\alpha) = L(w^{now} + \alpha v_1)$ for example direction v_1 (as a function of α with fixed w^{now}, v_1)



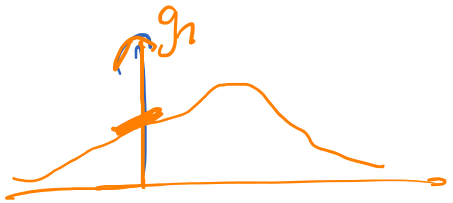
Contour lines



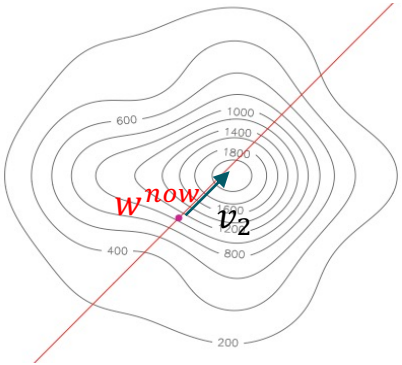
Loss landscape

$g_1(\alpha) = L(w^{now} + \alpha v_1)$
as a function of α

Visualization of directional derivatives

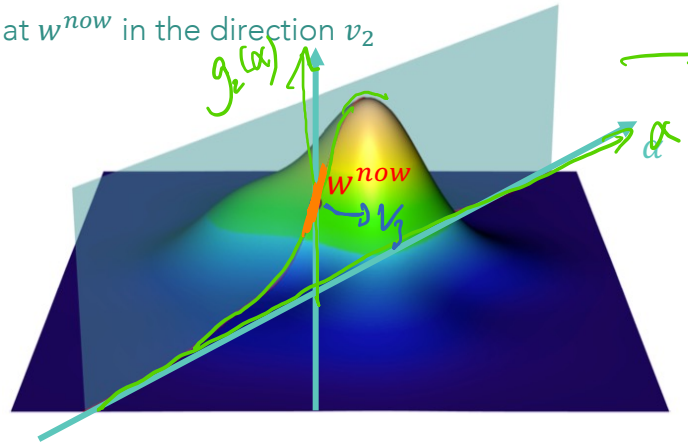


Plot of function $g_2(\alpha) = L(w^{now} + \alpha v_2)$ for example direction v_2 (as a function of α with fixed w^{now}, v_2)

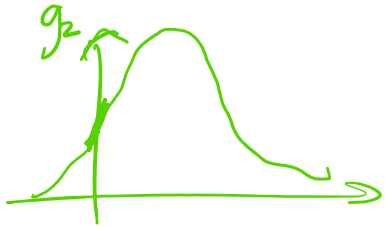


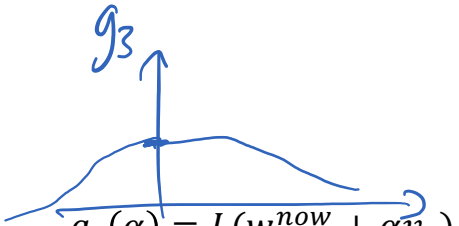
Contour lines

cut at w^{now} in the direction v_2



Loss landscape





$g_2(\alpha) = L(w^{now} + \alpha v_2)$
as a function of α

Plan for today

- Nonlinear regression
- Gradient descent for general losses
  - Warmup with 1d case: directions, stepsize, stopping criterion
  - Multi-d inputs:
    i. visualization of losses in 2d
    ii. negative gradient as steepest descent direction
      a. graphically using orthogonality to contour lines
      b. arithmetically using linear approximation
    iii. gradient descent for linear regression

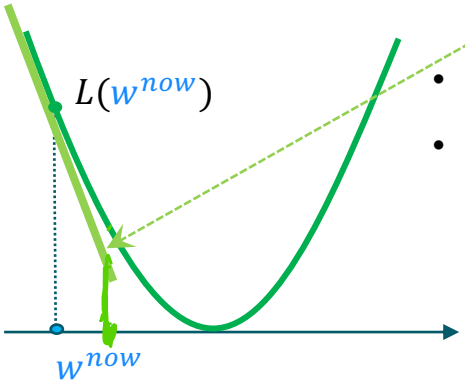
Steepest descent via linear approx. of the loss

- Reminder of general update formula: $w^{next} = w^{now} + \tilde{\eta}\, v$
- Assume that L is differentiable → best linear approximation of the loss w^{next} (for any direction v) is then

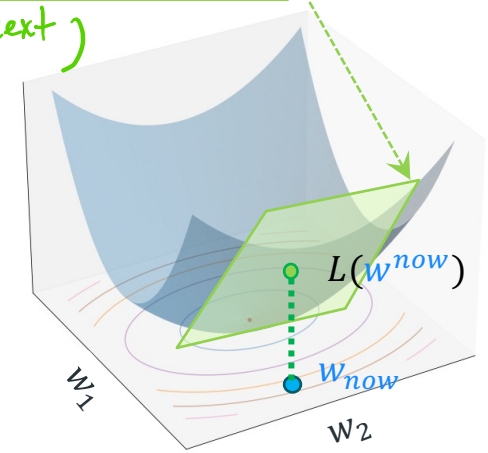
$$L(w^{next}) = L(w^{now} + \tilde{\eta}\, v) \approx L(w^{now}) + \tilde{\eta}\, \langle \nabla_w L(w^{now}), v \rangle$$

$L_{lin}(w^{next})$

Example in 1d



- linear function of v
- tangent planes to L at w^{now}



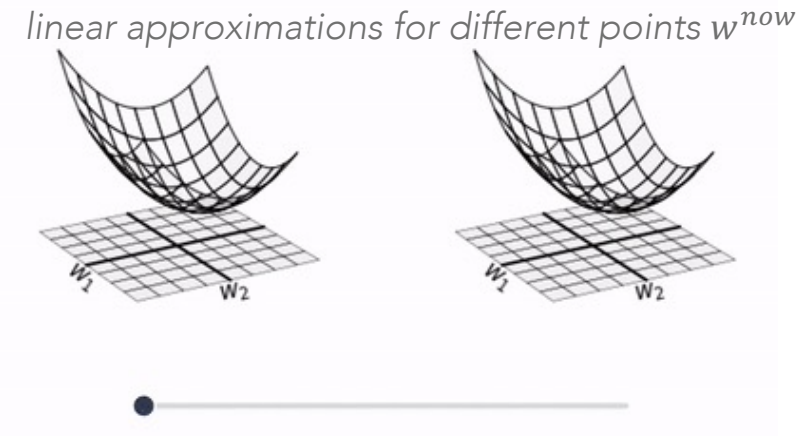
Example in 2d

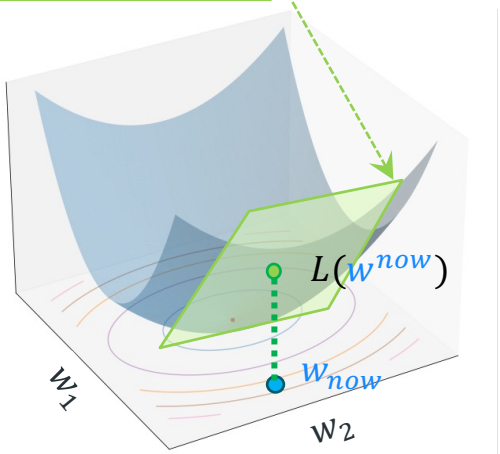
- Intuition: for small $\tilde{\eta}$, decrease of value of linear approx. (easier to compute) ≈ decrease on the true loss L

Steepest descent via linear approx. of the loss

- Reminder of general update formula: $w^{next} = w^{now} + \tilde{\eta}\, v$
- Assume that L is differentiable → best linear approximation of the loss w^{next} (for any direction v) is then

$$L(w^{next}) = L(w^{now} + \tilde{\eta}\, v) \approx \boxed{L(w^{now}) + \tilde{\eta}\, \langle \nabla_w L(w^{now}), v \rangle}$$





Example in 2d

- Intuition: for small $\tilde{\eta}$, decrease of value of linear approx. (easier to compute) ≈ decrease on the true loss L

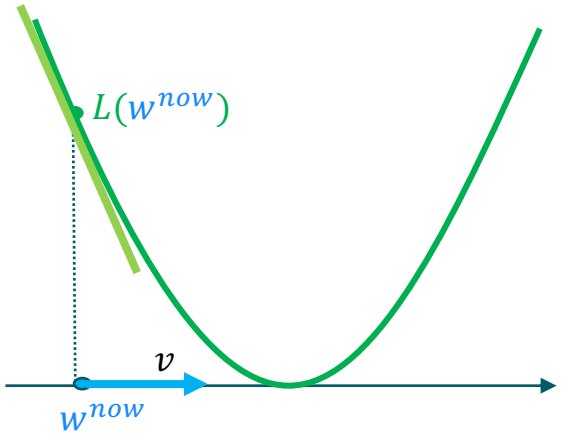
Source (left): https://github.com/jermwatt/machine_learning_refined

Steepest descent via linear approx. of the loss

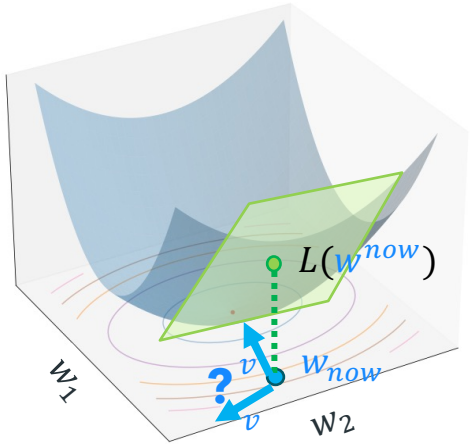
On linear approximation/tangent plane $L(w^{now}) + \tilde{\eta}\,\langle \nabla L(w^{now}), v \rangle$

in which direction (unit vector v) does the function decrease the fastest?

Example in 1d: $v = -\text{sign}(L'(w^{now}))$



Example in multi-d: $v = ?$



Visualization in 2d

For $\tilde{\eta}$ small enough, guaranteed that $L(w^{next}) < L(w^{now})$ slide 45

Steepest descent via linear approx. of the loss

On linear approximation/tangent plane $L(w^{now}) + \tilde{\eta}\,\langle \nabla L(w^{now}), v\rangle$

in which direction (unit vector v) does the function decrease the fastest?

- Derivation of steepest descent direction on tangent plane for $d > 1$:

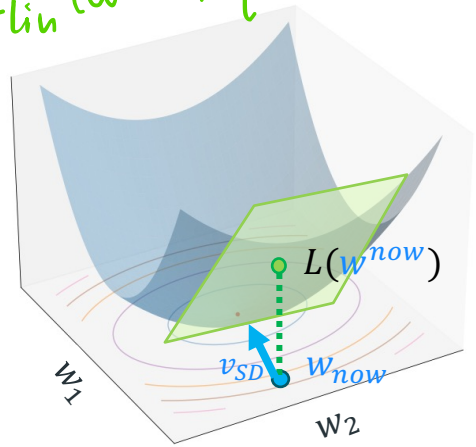
unit vector

$$v_{SD} = \text{argmin}_{||v||=1}\, L(w^{now}) + \tilde{\eta}\,\langle \nabla L(w^{now}), v\rangle = \text{argmin}_{||v||=1} L_{lin}(w^{now} + \tilde{\eta} v)$$

$$= \text{argmin}_{||v||=1} \langle \nabla L(w^{now}), v\rangle = -\frac{\nabla L(w^{now})}{||\nabla L(w^{now})||}$$

w^{next}

→ negative gradient points in the steepest descent direction of linear approximation/tangent plane!



Visualization in 2d



Iterative procedure on a high-level – multiple $d > 1$

General iterative algorithm to minimize $L(w)$

Start at initial w^{start}

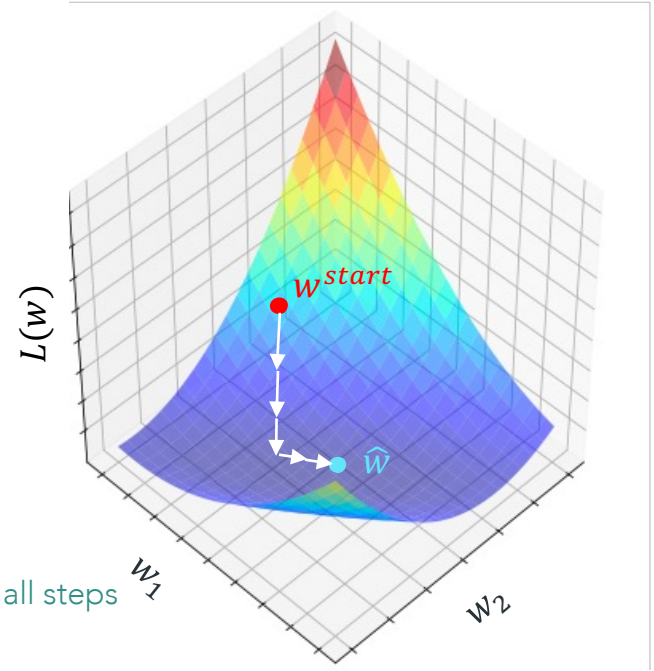
how far to move, can depend on step

update direction

At each step $w^{next} = w^{now} + \tilde{\eta} \cdot \frac{-\nabla L(w^{now})}{||\nabla L(w^{now})||}$

If e.g. $|L(w^{next}) - L(w^{now})| > \epsilon$, set $w^{now} = w^{next}$ & continue

Else, stop and output $\hat{w} = w^{now}$



Q1: Which direction v? → negative direction gradient

Q2: How far ($\tilde{\eta} > 0$) to go in that direction?

may be constant for all steps

→ Again choosing $\tilde{\eta} = \eta||\nabla L(w^{now})||$ for some step size η,

Q3: When to stop? → e.g. when $|L(w^{next}) - L(w^{now})| \leq \epsilon$

Gradient descent algorithm

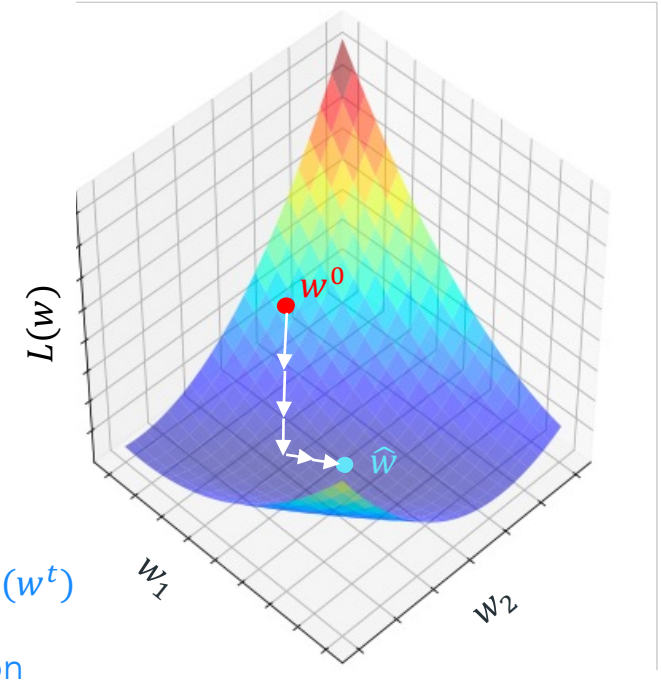
Gradient descent algorithm to minimize $L(w)$

Start at initial w^0

At each step t, $w^{t+1} = w^t - \eta \nabla_w L(w^t), t \leftarrow t+1$ (for simplicity, fixed for all t)

If e.g. $|L(w^t) - L(w^{t-1})| > \epsilon$, continue (can use $\|\nabla L(w^t)\| \leq \varepsilon$)

Else stop and output $\hat{w} = w^t$



Linear approximation for the loss value at step $t+1$

$$L(w^{t+1}) = L(w^t - \eta \nabla L(w^t)) \approx L(w^t) - \eta \langle \nabla L(w^t), \nabla L(w^t) \rangle < L(w^t)$$

→ the negative gradient direction is a descent direction

for small enough stepsize η (see lecture notes Theorem 5.2.)

More rigorously using Taylor expansion

BONUS

1. Multivariate Taylor expansion reads $L(w^{t+1}) = L(w^t) - \eta||\nabla L(w^t)||_2^2 + \xi(-\eta\nabla L(w^t))$
2. By definition of (directional) derivative for any vector v we have $\lim_{\eta\to 0} \frac{L(w+\eta v)-L(w)}{\eta} - \langle \nabla L(w), v\rangle = 0$
3. Hence plugging in $w = w^t$, $\mathrm{v} = -\nabla L(w^t)$ and remembering $w^{t+1} = w^t + \eta v$ we see that the remainder term satisfies $\lim_{\eta\to 0} \frac{\xi\left(-\eta\,\nabla L(w^t)\right)}{\eta} = 0$ and by definition of convergence,

   for any ϵ, there exists η small enough, s.t. $|\xi\left(-\eta\,\nabla L(w^t)\right)| \le \epsilon\,\eta$
4. Now pick $\epsilon < \left|||\nabla L(w^t)||\right|_2^2$. Then by the step above, there exists η that yields

$$L(w^{t+1}) \le L(w^t) - \eta\left|||\nabla L(w^t)||\right|_2^2 + \epsilon\,\eta < L(w^t)$$

the negative gradient direction is a descent direction (for small enough stepsize)

MORE TOMORROW…

Where does gradient descent converge to?



For a general function $L(w)$ that has a global minimizer w_{min} .

What does w^t converge to if we let $t \to \infty$ and choose small enough stepsize η?

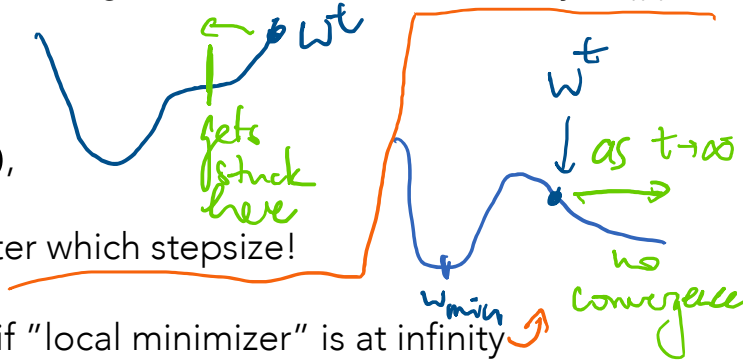
(A) Global minimum w_{min} (B) A local minimum (C) A stationary point (D) It depends

Solution: (D) Could find stationary point $\hat{w}$ such that $\nabla L(\hat{w}) = 0$, but in general that's **not** necessarily w_{min}!

- It might also not be a local minimum but just a saddle point

→ Even though we never increase the loss value $L(w^{t+1}) \leq L(w^t)$,

once we hit a point w^t with tiny $||\nabla L(w^t)||$ we're "stuck", no matter which stepsize!



- If you initialize wrongly, it might not find any stationary point if "local minimizer" is at infinity

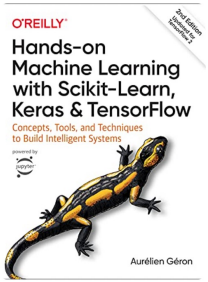
What you can do now…

- fit linear and nonlinear functions to your data using a gradient method
- derive the gradient descent update for parameterized functions on differentiable losses

Tomorrow:

- Other gradient-based methods (including stochastic gradient method)
- Convexity and optimality of stationary points
- How does this depend on # samples?

References



- Lecture notes Chapters 2 and 5
- peers found helpful: Aurelien Geron, Hands-on Machine Learning, Chapter 4
- Nocedal, Wright: Numerical Optimization

some picture sources

- from interactive demos: https://fa.bianp.net/teaching/2018/COMP-652/
- animations: https://github.com/jermwatt/machine_learning_refined